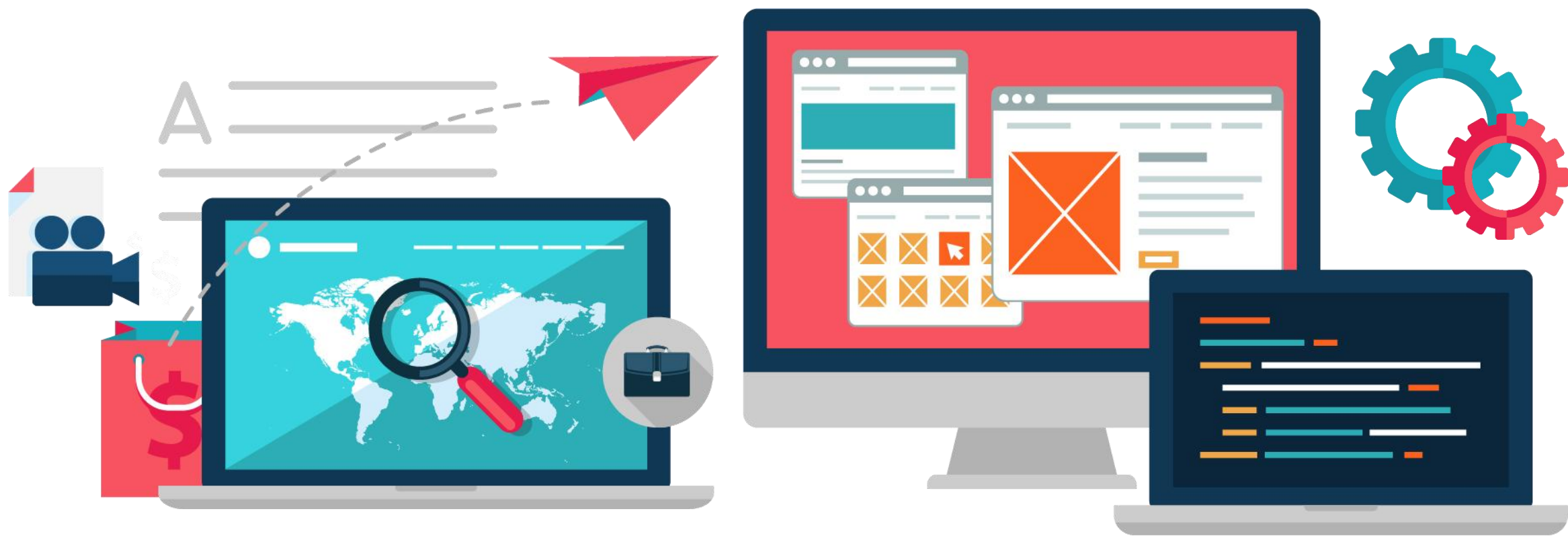


面向对象程序设计

Object Oriented Programming





类型和变量



01

抽象数据类型(ADT)

02

C++语言中的类型

03

声明和定义

04

声明和定义的使用原则

05

存储空间与作用域

06

表达式



抽象数据类型 (Abstract Data Type)

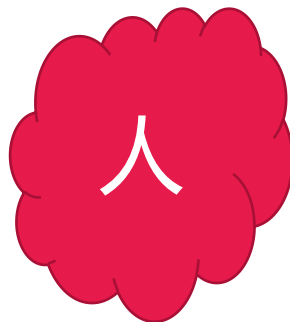
一个ADT：一个**数学模型**+可施加其上的**操作集合**。
(**类型名称**，**数据集**，**数据间的关系**，**操作集**)

- 与具体表示无关
- 与现实世界无关
- 任意性和无穷性

表示

```
a ADT :  
name {  
    数据:  
    关系:  
    操作:  
}
```

```
Person {  
    数据: age  
    关系: age >= 0  
    操作: birth  
           grow  
           dead  
}
```



```
Person {  
    数据: knowlege  
           fatigue  
    关系: knowlege > 0  
           fatgure >= 0  
    操作: study  
           sleep  
           play  
}
```



抽象数据类型 (Abstract Data Type)

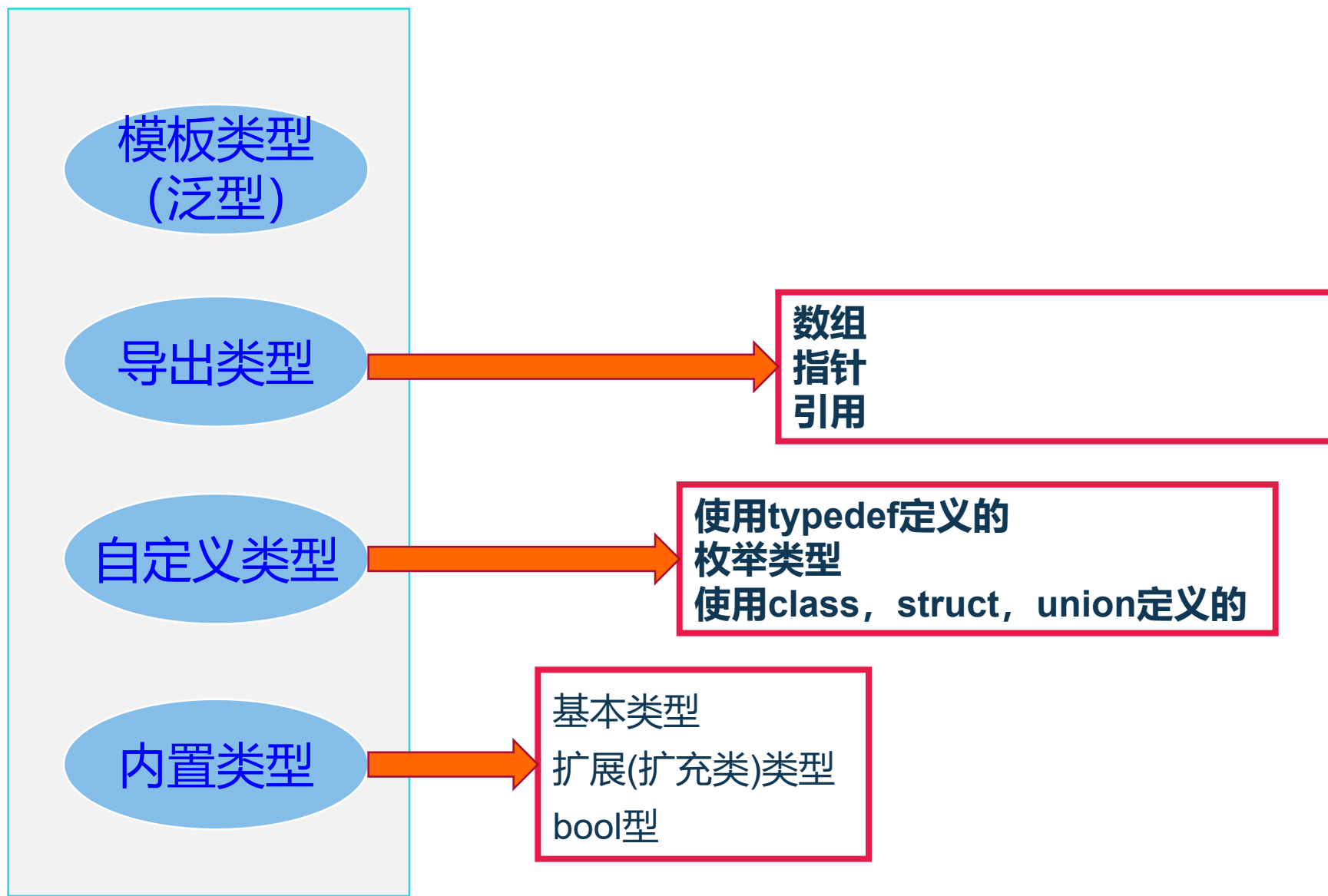
```
Person {  
  数据: age  
  关系: age >= 0  
  操作: birth  
        grow  
        dead  
}
```



```
整数 {  
  数据: number  
  关系: 0 is a Integer  
        success(...(0)..)=number  
        或 previous(...(0)..)=number  
  操作: success  
        previous  
}
```



- **基本类型**
- **自定义类型**
- **泛型 (generic type) : 以类型为参数的类型(参数化的类型)**
- **元类型及元对象(meta type/meta object):
各类型的类型 (C++暂不支持)**



- 基本类型: char , wchar_t(宽字符), int, float, (void)
- 基本类型的扩展: <类型修饰> 基本类型
 - 类型修饰: short、long、signed、unsigned、double
 - 例: long int, unsigned long int, double float, long double, ...
- bool型: 非零=true, 零=false
- C++11新增类型: 如 long long , 编译期推导类型 (auto, decltype)
例
 - auto: int a = 10; auto b=a+99;
 - decltype: int a = 10; decltype(sqrt(a)) b=sqrt(a+99)+1;
 - template<typename A,typename B>
auto multiply(A a, B b)->decltype(A*B)
{ return a*b; }

■ auto与decltype的区别

```
int num = 5;
```

auto aNum = num; // aNum的类型是int，因为它是根据num的值推导出来的

decltype(num) dNum = num; // dNum的类型也是int，但它是根据num这个表达式（而不是它的值）推导出来的，decltype会保留表达式的所有类型特征，包括是否是引用、是否是常量等，例如：

```
const int& constRef = num;
```

auto bNum = constRef; // bNum的类型是int，忽略const和引用，是独立的新变量

decltype(constRef) anotherConstRef = num; // anotherConstRef的类型是const int&，与constRef相同

在这个例子中，auto和decltype都推导出了int类型，但decltype更准确地保留了constRef的引用和常量特征。

综上所述，auto和decltype都是C++中用于类型推断的关键字，但它们在使用方式和目的上有所不同。auto主要用于简化变量声明和函数的类型推断，decltype主要用于查询获取表达式的精确类型，在处理引用、常量类型或模板编程时非常有用。



使用typedef的自定义类型

- typedef可以用来定义已有类型的同义词，可以出现在程序中类型名可出现的任何位置
- 定义类型格式：typedef <已知类型> <新类型>
- 定义函数类型格式：typedef ReturnType (*新类型)(参数列表);
- **typedef本质上没有增加新类型**

// 例1

```
typedef unsigned char  UCHAR
typedef unsigned long  DWORD
typedef unsigned int   uint;
typedef double wages;
```

UCHAR ch = 'a'; 等价于 unsigned char ch = 'a';
DWORD n = 99; 等价于 unsigned long n = 99;
uint num = 10; 等价于 unsigned int num = 10;
wages w = 5888.88; 等价于 double w = 5888.88;

- **简化复杂的类型定义，使其更易理解**
- **强调使用类型的目的，弱化或隐藏特定类型的实现**

// 例2:

```
int f(int,int) { }
int g(int ,int) { }
typedef int (*MyFuncType)(int,int);
MyFuncType f1=f;
MyFuncType f2=g;
f1(1,2); f2(3,4);
int YourFunc(MyFuncType ff)
{ return ff(88,99); }
YourFunc(f); YourFunc(g);
```



- C++11可以用using来创建类型别名，完全可以替代typedef，更加清晰易读

// 例1

```
using UCHAR = unsigned char;
using DWORD = unsigned long;
using uint = unsigned int;
using wages = double;
UCHAR ch = 'a';    DWORD n = 99;
uint num = 10;    wages w = 5888.88;
```

// 例3

```
int (*f1)(int, int) = f;    //函数指针变量f1
int (*f2)(int, int) = &g; //函数指针变量f2
int (*funArray[2])(int, int) = {f, g}; //数组变量
using FunArrayT = int (*[2])(int, int); //数组类型
using FunArrayT = MyFuncType[2]; //数组类型
```

// 例2:

```
#include <functional>
int f(int,int) { }
int g(int ,int) { }
using MyFuncType = int (*)(int, int);
using MyFuncType =
    std::function<int(int, int)>;
MyFuncType f1=f; // 函数名隐式转换为指针
MyFuncType f2=&g; // 显式取地址，也可以
f1(1,2);
f2(3,4);
int YourFunc(MyFuncType ff)
{ return ff(88,99); }
YourFunc(f); YourFunc(g);
```

- 如果一个变量只有几种可能的值，可以定义为枚举类型。所谓枚举就是指将变量的值一一列举出来，变量的值只能在列举出来的值的范围内。

- 例如，文件打开的状态可能会有三种：输入、输出和追加
- `enum open_modes {input, output, append};`

C++98: `enum` 枚举类型名 {枚举值列表};

- 枚举值必须是整数
- 第一个枚举值，缺省为0
- 后一个枚举值，若没有指定，则为前一个枚举值+1

例:(C++98)

```
enum WeekDay  
{MON=1,TUR,WED,THU,FRI,SAT, SUN=0};
```

```
int main( )
```

```
{  
    WeekDay day1,day2;  
    day1=SUN,day2=THU;  
    cout<<day1<<endl;  
    int num = day2+100;  
} // day1 = num; fail C中ok
```

C++98下的说明:

1. 内部就是int
2. int与枚举量可随意转换，不安全
3. 同名枚举值的问题

```
enum Dir { LEFT,RIGHT};
```

```
enum State { RIGHT,FAILED};
```

RIGHT==0 还是 RIGHT==1 ?



C++11: `enum class` 枚举类名 { 枚举值列表 };

- 枚举值可以指定类型、指定数值
- 枚举值不再允许与int随意转换、比较
- 使用时必须指明scope(即枚举类名)

◆ 例 (C++11) :

```
enum class AAA : unsigned int {  
    A    = 'a',  
    EF = 120  
};  
  
int main( ) {  
    cout<< (int)(AAA::A)<<endl;  
    cout<< (int)(AAA::EF)<<endl;  
    return 0;  
}
```



c++中，class/struct，除了默认的控制访问和继承方式不同之外，完全一致

◆ 格式:

```
class <Name> {  
    <members>  
};
```

◆ 例:

```
class Dog {  
public:  
    void Eat( int num);  
private:  
    int weight;  
}; //不要忘了; 号
```



```
enum Dir { LEFT, RIGHT };
```

缺点：名字可能冲突

优点：不用与int类型相互转换

```
int f( Dir dir) { return dir+1; }  
f(RIGHT ); //f(0); fail
```

```
enum class Dir { LEFT, RIGHT };
```

缺点：需要显式转换

优点：避免名字冲突

```
int f( Dir dir) { return (int)dir+1; }  
f((Dir)0);
```

```
class My {  
    public:  
        enum Dir { LEFT, RIGHT };  
    public:  
        ...  
};
```

```
int f(My::Dir dir ) { return dir+1; }  
  
f( My::RIGHT);
```



◆ 数组

例: `int a[5]; MyClass objs[3];`

◆ 指针

例: `int * p = 0; MyClass * pobjs[4];`
`int **pp = &p;`

◆ 引用:

例: `int a = 100; int & b = a;`

右值引用(C++11):

`int x = 42; // x 是一个左值`

`int& lref = x; // 左值引用, 绑定到左值x 上`

`int&& rref = 42; // 右值引用, 绑定到一个字面量右值上`

左值: 具有持久状态的表达式, 可以取地址。

右值: 临时对象或字面量, 或即将被销毁的对象, 没有持久地址。



• 什么是变量

```
int x = 1; float y = 3.4; char c = 'a' ;
```

- 变量是用来保存数据的程序元素；变量是指程序中可以操作的有名字的内存单元，是内存单元的别名
- 给一个变量赋值就是把值写入它代表的内存单元中，而取一个变量的值就是读取其内存单元中存放的值
- C++中的每一个变量都有特定的类型，该类型决定了变量的内存大小和布局、能够存储于该内存中的值的取值范围以及可应用在该变量上的操作集，编译时会作类型检查
- 变量名是左值表达式，变量本身是一个实体



• 关于变量名

- 变量名，即变量的标识符，可以由字母、数字和下划线组成。变量名必须以字母或下划线开头，并且区分大小写字母
- 变量名的长度没有限制，C++ 标准没有规定标识符的最大长度，只要求实现（编译器+链接器）能够支持“足够长”的标识符（例如C++17规定外部标识符必须至少能区分前1024个字符）
- 关键字不能用作变量名
- 一些操作符的替代名不能用作变量名（and、or、not、bitand、xor等作为关键字保留）
- 标准库中标识符不要用作变量名（__reserved、assert、NULL、EOF、vector、sort、count等）
- 变量名一般用小写字母，应能提示其在程序中的目的和用法



- 声明：解释说明一个编译单元中一个名字的含义和属性。

<code>extern int a;</code>	<code>// 声明外部整型变量a</code>
<code>extern const int c;</code>	<code>// 声明外部整型常量c</code>
<code>int f(int);</code>	<code>// 声明函数f</code>
<code>struct S;</code>	<code>// 声明结构S</code>
<code>typedef int Int;</code>	<code>// 声明类型 Int</code>
<code>extern X anotherX;</code>	<code>// 声明外部变量anotherX</code>
<code>using N::d;</code>	<code>// 声明名字d</code>

- **定义**：一个声明同时也是一个定义，当：
确定的函数体、确定的类内容、有具体值、
确定存储空间、确定别名等时

```
int a;                // 定义a
extern const int c = 1; // 定义c
int f(int x) { return x+a; } // 定义 f 和 x
struct S { int a; int b; }; // 定义S、S::a、和S::b
struct X {            // 定义 X
    int x;            // 定义非静态数据成员x
    static int y;      // 声明静态数据成员y
    X(): x(0) { }      // 定义X的构造函数
};
int X::y = 1;          // 定义 X::y
enum { up, down };    // 定义up 和 down
namespace N { int d; } // 定义N 和 N::d
namespace N1 = N;      // 定义N1
X anX;                 // 定义anX
```



- **C++**区分声明和定义的根本目的是为了支持分离编译，从而实现高效的模块化开发和代码组织。

• 变量的声明和定义

- 变量的声明用于表明变量的类型和名字，而不必分配存储空间,不必告知其具体位置
- 变量的定义用于为变量分配存储空间，还可以为变量指定初始值
- 定义是声明，因为定义变量时声明了它的类型和名字
- 声明未必是定义，可以通过使用extern关键字声明变量名而不定义它
- 只要声明存在初始值就被认为定义，不管是否有extern



- 类型的声明和定义:
- 函数的声明和定义:
- 变量的声明和定义:

//类型声明(前置声明)

```
class Meat;  
typedef unsigned int uint;
```

//类型定义

```
class Dog {  
public:  
    void Drink( );  
    void Eat(Meat * m);  
private:  
    int num;  
    int val;  
};
```

//函数(原型)声明

```
int MyFunc( int );  
int YourFunc( );
```

//函数定义

```
int MyFunc( int n )  
{  
    return 2*n;  
}
```

//变量的声明

```
extern int a;  
extern Dog aDog;
```

//变量的定义

```
int a; //无初始化, 不建议  
float b = 2.5;  
Dog aDog;
```

- 就近原则(现使用，现定义):
- 先声明后使用原则:
- 单一定义规则:

//在C语言中(C89/90)

```
int main( )
{
    int i, j, k;
    for(i=0;i<10;++i) {
        for(j= 1;j<20;++j) {
            k = f(i,j);
            // use k
        }
    }
}
```

//在现代语言中(含C++)

```
int main( )
{
    for(int i=0;i<10;++i) {
        for(int j= 1;j<20;++j) {
            int k = f(i,j);
            // use k
        }
    }
}
```

```
//a.cpp
int count =1;
int main()
{
}
```

```
//b.cpp
extern int count;
void f( )
{
    count = 10;
}
```

```
//dog.h
class Bone;
class Dog
{
public:
    void Eat(Bone * p);
    //void Eat(Bone b);      ok
    //void Eat(Bone b) { }  fail
    // error: incomplete type
};
```

① 在一个编译单元中，同一个标识符（如变量、函数、类名等）只能被定义一次，但声明可以多次(否则出现编译期重定义错误)；

② 在同一个项目的所有多个编译单元中，对于大多数实体（非内联函数、非内联变量等），只能有一个定义(否则出现链接期重定义错误)。对于类/枚举/内联函数/内联变量/模板等，每个使用它的翻译单元必须有定义且所有定义必须完全相同。

```
//反例1
//a.cpp
int count =1;
int count =1;
int main()
{
    ...
}
```

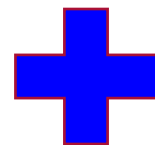
```
//反例2-a.cpp
#include "a.h"
int main( )
{
    count = 10;
    f( count );
}
```

```
//反例2-a.h
int count =1;
int f( int n) {
    return 1;
}
```

现在反例2能编译通过吗？


```
//反例2-a.cpp
#include "a.h"
int main( )
{
    count = 10;
    f( count );
}
```

```
//反例2-a.h
int count =1;
int f( int n) {
    return 1;
}
```



```
//反例2-b.cpp
//b.cpp
#include "a.h"
void g( ) {
    int n = f(count);
}
```

?

```
//反例2-a.h
#ifndef AH
#define AH
int count =1;
int f( int n) {
    return 1;
}
#endif
```

添加包含警戒能解决吗?

```
//a.cpp
#include "a.h"
int main( )
{
    count = 10;
    f( count );
}
int count = 1;
int f( int n) {
    return 1;
}
```

函数f的定义，可放在a.cpp中，也可放b.cpp中，但只能选择其中一种

```
//a.h
#ifndef AH
#define AH
extern int count;
int f( int n);
#endif
```

count的定义可放在a.cpp中，也可放b.cpp中，但只能选择其中一种

```
//b.cpp
#include "a.h"
void g( ) {
    int n = f(count);
}
```

结论：

1. 头文件中可放声明，以及类定义，但不应放变量定义和函数定义
2. 函数定义及变量定义，只能在cpp文件中定义一次

注：本例中，头文件为a.h，表示的是a.cpp文件对外提供的功能，所以从规范上讲，两个定义放入a.cpp最好。



变量的定义:

C++98: 格式: [`<存储类>`]`<类型>` `<变量名表>` [`=初始化表`];

C++11: 新增: [`<存储类>`]`<类型>` `<变量名表>` [`{初始化表}`];

存储类: 自动的(auto)、静态的(static)、寄存器的(register)和外部的(extern)

// 从 C++11, auto不再是存储类说明符, register被弃用, 且C++17被移除。

//C++98

```
int state=1,age,weight=10,val=weight;
```

```
static long count; // 函数外的static变量, 其中static表示文件级变量,  
                  // 只在本文件中有效(C++11应使用匿名名字空间取代)
```

```
auto float r=0.5; //auto表示自动变量, 其含义与c++11不同
```

//c++11

```
int state=1,age,weight=10,val=weight,val2{weight};
```

```
float r =0.5;
```

```
auto val3=r; //auto表示val3的类型为自动推导类型
```



在C++中，一个程序占用的内存区一般分为以下几部分

- 1.全局、静态变量数据区：存储全局变量及静态变量（包括全局静态变量和局部静态变量），分配该区时内存全部清零
- 2.常量数据区：存储程序中的常量字符串等
- 3.代码区：存储程序的代码，即程序中各个函数的代码块
- 4.栈区：存储局部变量，如函数中的变量等；由系统自动分配，分配栈区时不处理内存，即变量取随机值
- 5.堆区：存储动态产生的数据，由程序员动态分配和回收，分配堆区时不处理内存

程序

代码区

常量数据区

全局、静态变量数据区

栈区

堆区

程序区

```
int n=100;  
int f(int n) {  
    static int m=8;  
    return m+n;  
}  
int main () {  
    int i=10;  
    for(int j=i;j<10;++j) {  
    }  
    std::cout<< "吉林大学" ;  
}
```



- 生存期是指变量在内存中存在的时间。C++所有的变量都有有限的生存期，它们从被定义的那一刻起存在，并在某一刻消失，最迟也要在程序终止时消失。表示的是“存活状态”
- 变量的生存期取决于它的存储类
- 变量拥有的另一个属性是作用域，作用域是指变量在程序中的作用（有效）范围。在变量的作用域中，可以合法地引用它，在变量的作用域之外，就不能引用它的名称，否则会导致一个编译错误。注意，变量在其作用域之外仍然存在，只是不能根据名称来引用它。表示的是“可见性”和“可用性”
- 变量的作用域可以是全局作用域、文件作用域、块作用域、类作用域、名字空间作用域等。



全局作用域:

文件作用域:

块级作用域:

类级作用域:

名字空间作用域:

```
extern int global;
int state=1;
static int filevar=99;
int f(int n) {
    static int sa =9;
    int m = sa;        //C++98下, 等价于auto int m=sa;
    auto k = sa+8;     //只在C++11下有效
    return sa+n;
}
struct A {
    void fff( ) { }
    int val;
};
```

```
#include <iostream>
int main() {
    int k=10;
    A aA;
    aA.val = 2;
    aA.fff();
    std::cout<< "输出" ;
    return 0;
}
```



存储空间和作用域(例)

```
#include <iostream>
using namespace std;
/// 全局数据区-num (全局作用域)
int num = 999;
/// 全局数据区-num2 (文件级作用域)
static int num2 = 8;

void func(int a,int b,int c) ///abc压入栈区
{
    /// 全局数据区-n (块级作用域)
    static int n = 0;
    ///result 栈区(块级作用域)
    int result = (a+b+c)*(++n);
    ///字符串"Result= "全局数据区 (常量)
    cout<<"Result= "<<result<<endl;
}
```

```
int main() {
    int a = 10;        /// 栈区
    static int b = 20; /// 全局数据区
    func(a,b,20);
    func(a,b,20);

    /// 变量p本身, 分配在栈区, 其指向
    /// 堆区中的一个地址
    int * p= new int(55);
    ...
    delete p;
}
```




- 局部变量及其作用域和生存期
- 局部变量是指在一个块中声明的、即在一对花括号中声明的变量，也称为自动变量
- 局部变量的作用域是从它声明的位置开始，直到所在块的结束(块作用域)
- 局部变量从所在块执行起存在，直到所在块执行结束时自动消失。每次执行到包含局部变量声明的语句块时，就创建一个新的变量。如果为局部变量指定了初始值，则每次创建该变量时都会重新初始化它，如果没有为局部变量指定初始值，则它的初始值是不确定的



- 全局变量及其作用域和生存期
- 全局变量是指在所有块和类外部声明的变量
- 全局变量的作用域从它声明的位置开始至源文件结束（文件作用域），即全局变量能够被位于其定义位置之后的本源文件中的所有函数共用
- 全局变量从程序开始执行起存在，直到程序结束时消失。全局变量的初始化在main()开始执行之前进行，如果没有为全局变量指定初始值，在默认情况下初始化为0，并转换为该变量适用的类型
- 全局变量允许用extern将其作用域扩展至其它源文件



- 静态变量及其作用域和生存期
- 静态变量的作用域可以是文件作用域，也可以是块作用域、类作用域，因此静态变量可以是局部静态变量，也可以是全局静态变量
- 静态变量从程序开始执行到声明它的语句起存在，直到程序结束时消失。如果没有为静态变量指定初始值，则它总是会自动初始化为0，并转换为该变量适用的类型
- 全局静态变量与全局变量的区别
- 全局静态变量的作用域局限于所在源文件，不允许用extern将其作用域扩展至其它源文件



- 表达式是由操作数和运算符组成的序列，用于计算一个值，并可能产生副作用（如修改对象、输入 / 输出等）。
- 操作数可以是字面量（如 42、3.14）、变量（如 x）、函数调用（如 f()）、lambda 表达式（如 `[] { return 0; }`）等。
- 运算符包括算术运算符（+、-、*、/）、关系运算符（==、!=）、逻辑运算符（&&、||）、位运算符（&、|）、赋值运算符（=）等。
- **每个表达式都由两个独立的属性来表征：类型和值类别。**
- 类型：表达式计算结果的数据类型，如 int、double、void 等。类型决定了表达式可参与的操作、内存布局及语义约束。
- 值类别：按表达式计算结果值的性质分类（如左值、纯右值、将亡值等）。
- **类型描述：值是什么，值类别描述：值如何使用**，两者相互独立但协同作用。例如，int 类型的表达式既可以是左值（如变量 x），也可以是纯右值（如字面量 42）。



表达式的特别说明

- for循环中的表达式： 初值循环前执行1次，条件每次循环前，步进每次循环后
for(初值表达式; 条件表达式; 步进表达式) for(;;) for(f(),g(),h();;)
- 逗号表达式：
例： `x=100, y=x-1000;`
- 条件表达式：
例： `if (表达式1 && 表达式2) {...}`
- 赋值表达式
例： `x=y=100+f(5); (x=y)=100;` 从右到左结合，返回其左侧操作数的引用
- 左值表达式：能够作为=号的左侧表达式，也可以作为右侧表达式
- 右值表达式：只能作为=号的右侧表达式
- (C++11) lambda表达式
 - 通用格式： `[capture] ( params ) mutable exception attribute -> ret { body }`
 - 简化格式1： `[capture] ( params ) -> ret { body }`
 - 简化格式2： `[capture] ( params ) { body }`
 - 简化格式3： `[capture] { body }`

`x = <表达式1> ? <表达式2> : <表达式3>;`

先求解表达式1，若为true，则求解表达式2，将表达式2的值赋给x。若为false，则求解表达式3，将表达式3的值赋给x。



附:lambda表达式简介

Lambda表达式代表的是一个匿名(无名)的函数。

Lambda表达式语法格式 (4种) :

1. `[ capture ] ( params ) mutable exception attribute -> retType { body }`
2. `[ capture ] ( params ) -> retType { body }`
3. `[ capture ] ( params ) { body }`
4. `[ capture ] { body }`

其中

(1) 是完整的 lambda 表达式形式;

(3) 是省略了返回值类型的 lambda 表达式, 函数返回值类型由body自动推演出来。

(4)省略了参数列表, 类似于无参函数 `f()`。

`mutable` 修饰符表明body内的代码可以修改被捕获(capture)的变量, 并且可以访问被捕获对象的 `non-const` 方法。`exception` 说明 lambda 表达式是否抛出异常(`noexcept`), 以及抛出何种异常, 类似于`void f() throw(X, Y)`。`attribute` 用来声明属性。`retType`指明返回类型, 可以明确地指定, 如`int`, `bool`,自定义类型等, 也可以是`decltype`推演类型。

`capture` 指定了body中可以访问的外部变量列表, 具体解释如下:

`[a,&b]`以传值的方式捕获a变量, 以引用的方式捕获b变量。

`[this]` 以值的方式捕获 `this` 指针。

`[&]` 以引用的方式捕获所有的外部自动变量, 可修改外部量。

`[=]` 以传值的方式捕获所有的外部自动变量, 不可修改外部量。

`[]` 不捕获外部的任何变量。



附:lambda表达式例

```
#include <iostream>
using namespace std;
void show(int a,int b,int c) {
    cout<<"a="<<a<<"\t";
    cout<<"b="<<b<<"\t";
    cout<<"c="<<c<<endl;
}
```

```
int main() {
    int a = 10;    int b = 20;    int c = 40;
    auto func1 = []( int & n ) { return n + 5; };
    show(a,b,c);    /// 输出 a= 10 b=20 c= 40
    show(a,b,func1(c)); /// 输出 a= 10 b=20 c= 45
    show(a,b,c);    /// 输出 a= 10 b=20 c= 40
    auto func2 = []( int & n ) { return n += 5; };
    //auto func2 = []( int & n ) { return n += 5 + a; };
    //Error:不能访问a
    show(a,b,c);    /// 输出 a= 10 b=20 c= 40
    show(a,b,func2(c)); /// 输出 a= 10 b=20 c= 45
    show(a,b,c);    /// 输出 a= 10 b=20 c= 45
}
```

```
auto func2 = []( int & n ) { return n += 5; };
//auto func2 = []( int & n ) { return n += 5 + a; }; //Error:不能访问a
show(a,b,c);    /// 输出 a= 10 b=20 c= 40
show(a,b,func2(c)); /// 输出 a= 10 b=20 c= 45
show(a,b,c);    /// 输出 a= 10 b=20 c= 45

auto func3 = [=]( int & n ) { return n += a+b; };
//auto func3 = [=]( int & n ) { a=b; return n += a+b; }; /// Error:不能修改a
show(a,b,c);    /// 输出 a= 10 b=20 c= 45
show(a,b,func3(c)); /// 输出 a= 10 b=20 c= 75
show(a,b,c);    /// 输出 a= 10 b=20 c= 75

auto func4 = [a]( int & n ) { return n += a; };
//auto func4 = [a]( int & n ) { return n += a+b; };    /// Error:不能访问b
//auto func4 = [a]( int & n ) { a= 99; return n += a; }; /// Error:不能修改a
show(a,b,c);    /// 输出 a= 10 b=20 c= 75
show(a,b,func4(c)); /// 输出 a= 10 b=20 c= 85
show(a,b,c);    /// 输出 a= 10 b=20 c= 85

auto func5 = [&b,a]( int & n ) { ++b; return n += a+b; };
show(a,b,c);    /// 输出 a= 10 b=20 c= 85
show(a,b,func5(c)); /// 输出 a= 10 b=21 c= 116
show(a,b,c);    /// 输出 a= 10 b=21 c= 116

auto func6 = [&b,a]( int n ) { ++b; return n += a+b; };
show(a,b,c);    /// 输出 a= 10 b=21 c= 116
show(a,b,func6(c)); /// 输出 a= 10 b=22 c= 148
show(a,b,c);    /// 输出 a= 10 b=22 c= 116

return 0;
}
```



本章结束